CodeBEAM 2026: DistErl over AMQP

James Aimonetti

2026-03-22

Outline

Hi and welcome

James Aimonetti (Eye Moe Net Tea)

- KAZOO Architect at 2600Hz/OOMA
- Erlanger since 2007

What is KAZOO?

- Telecom platform started in 2010
- Distributed telecom nodes (FreeSWITCH and Kamailio)
- Distributed control plane (KAZOO)
- Distributed data store (CouchDB)
- Message bus to tie it together (RabbitMQ)

What is AMQP (Advanced Message Queuing Protocol)?

- Network protocol
- Decouples the publisher (sender) from the consumer (receiver)
- Provides exchanges, queues, and flexible routing strategies
- TCP connection to broker, virtual channels for publishers and consumers
- Exchanges can route messages
  - directly to queues
  - fanned out to all queues
  - or use binding (consumer) and routing (publisher) keys for flexible reception

Erlang Distribution

- Connect to remote nodes
- Send and receive Erlang terms directly between nodes
- Process location is transparent (encoded in the PID)

Default DistErl

- Uses gen_tcp for transport
- Uses a separate program EPMD (Erlang Port Mapper Daemon)
  - Maintains a directory of running VMs on the server and what ports

```
> epmd -names
epmd: up and running on port 4369 with data:
name kazoo_apps at port 11500
name node1 at port 38957
name rabbit at port 25672
```

- Remote nodes ask EPMD how to reach an Erlang node
- Connected nodes over TCP, using cookies for authentication
- Fully connected mesh (or hub-and-spoke for "hidden" nodes)

Why not use EPMD-based DistErl?

- EPMD provides insight into your internal BEAM VMs, further attack surface
- Cookie-based auth in VMs isn't for security (read more from The EEF on how to secure EPMD)
- Mesh networking doesn't scale well (50 is an oft-repeated limit; 1500 if you have global infra to play with)
- EPMD complicates matters in docker/kubernetes
- Observing inter-node communication from the outside is challenging
- More firewall ports to manage

Extensible distribution

- Alternative carrier protocols are supported
- TCP/IP is common
- `uds_dist` provides distribution over Unix domain sockets for Sun Solaris 2
- Erlang drivers are complicated - we didn't touch this and just use TCP/IP
- Distribution modules provide well-defined callbacks

Distribution modules

- Details on finding other nodes
- Create a listen port (or an approximation)
- Connect to other nodes
- Perform the handshake and cookie verification
- `dist_util` makes a lot of this straight-forward

Why AMQP?

- In KAZOO clusters, RabbitMQ is already running
- Engineering and operation teams are familiar, existing tooling
- Single TCP connection to broker
- Publishes and consumes messages from other nodes
- Segment clusters using RabbitMQ virtual hosts on same broker

amqp_dist

- Erlang library:
  `https://github.com/2600hz/erlang-amqp_dist`

amqp_dist

Configure you AMQP broker(s)

```
[
  {amqp_dist, [
    {connections, [
        "amqp://guest:guest@localhost:5672"
      ]}
  ]}
].
```

amqp_dist

Using multiple brokers

```
["amqp://guest:guest@rabbitmq:5676/lx1"
,"amqp://guest:guest@rabbitmq:5676/lx2"
,"amqp://guest:guest@rabbitmq:5676/lx3"
]
```

- Tracks latency stats and will use "best" broker for inter-node connections

amqp_dist

Start some nodes

```
erl -pa _build/default/lib/*/ebin \
 -proto_dist amqp \
 -no_epmd \
 -name node01@your.host.com \
 -setcookie change_me \
 -config ./config/sys.config
```

Module architecture

- `amqp_dist`
  - Provides the callbacks for net_kernel
  - `listen(Name)` when VM brings up distribution, starts the whole shebang
- `amqp_dist_acceptor`
  - Handles broker connections and node heartbeats
  - Tracks new nodes joining/leaving
  - Receives when remote nodes want to connect
- `amqp_dist_node`
  - started after a node handshake is completed
  - handles inter-node message passing
  - Erlang terms are encoded using term_to_binary and base64 encoding before publishing

amqp_dist

Implements the net_kernel callbacks

- `listen/1` returns a "fake" socket record with amqp_dist_acceptor `pid()`
- `accept/1` spawns a process to handle incoming connection requests from remote Erlang nodes, notifies amqp_dist_acceptor
- `accept_connection/5` spawns a process to perform the handshake with the remote Erlang node
- `setup/5` spawns a process to handle inter-node communication via AMQP
- `select/1` determines if this dist module should handle connection setup

amqp_dist_acceptor

Handles broker connections, DistErl connection requests
Once the broker connection is established:

1. an AMQP channel is started
2. the exchange `amq.headers` is configured (for routing on attributes vs routing keys)
3. an exclusive queue is declared:
   `list_to_binary(["amqp_dist_acceptor-", atom_to_list(node()), "-", pid_to_list(self())]);`
   - may include the Label in queue name, if configured
4. the queue is bound to the exchange with a header argument
   `{«"distribution.ping"», bool, true}`
5. start consuming from the queue

amqp_dist_acceptor

When publishing a "node up":

```
,reply_to = QueueName
,headers = [{<<"distribution.ping">>, bool, true}
           ,{<<"node.start">>, timestamp, Start}
           ]
```

amqp_dist_acceptor

Handles two AMQP message types:

1. New node is present - receives the heartbeat message from other nodes
   - can configure `amqp_dist` to auto-connect
2. New node wants to connect

Potential issues

- Broker down: all nodes disconnected
  - KAZOO architecture already has handles for this too
  - Clustered rabbit possible, multi-broker works too
- Broker flow control
  - Rabbit can slow "busy" channels and connections
  - More noticable than TCP backpressure when its active
- Two hops introduces latency
- Throughput is capped at the broker

Benefits

- Access controls and vhosts
- Inter-node traffic is observable at the broker
- Piggy-backs on existing infrastructure and knowledge
- Flexible routing options (currently direct to queue) available
- Handles split-brain more gracefully
- Using modules like `ra` or `pg` more reliable
- Quite performant for our use

AltDistErl

- Alternative distribution modules are straight-forward to write!
- Plug in your favorite protocols to carry DistErl
- AMQP-based can simplify setup if you already have it in your infra

Thanks

- `amqp_dist`: `https://github.com/2600hz/erlang-amqp_dist`

Questions?